ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

Système de Réservation Hôtelière avec Django

Par : HADIDOU Hajar & ROUROU Rim & KHALD Hiba

Encadrant : ABOUABID Hamza

Projet présenté dans le cadre de la 3e année
EMSI Les Orangers
Année universitaire : 2024-2025

Table des matières

1 Introduction **3**
1.1 Contexte du projet . . . 3
1.2 Problématique . . . 3
1.3 Objectifs du projet . . . 3
1.4 Présentation de la solution . . . 4

2 Analyse des besoins **5**
2.1 Besoins fonctionnels . . . 5
2.1.1 Gestion des utilisateurs . . . 5
2.1.2 Gestion des hôtels . . . 5
2.1.3 Gestion des chambres . . . 5
2.1.4 Gestion des réservations . . . 6
2.2 Besoins non-fonctionnels . . . 8
2.2.1 Performance . . . 8
2.2.2 Sécurité . . . 8
2.2.3 Disponibilité . . . 8
2.2.4 Scalabilité . . . 8

3 Conception **9**
3.1 Architecture globale . . . 9
3.2 Modèle de données . . . 9
3.2.1 Entités principales . . . 9
3.2.2 Relations . . . 11
3.3 Interface utilisateur . . . 11
3.3.1 Pages principales . . . 11

4 Implémentation **12**
4.1 Environnement technique . . . 12
4.1.1 Technologies utilisées . . . 12
4.1.2 Dépendances principales . . . 12
4.2 Structure du projet . . . 12
4.3 Fonctionnalités clés . . . 13
4.3.1 Système de réservation . . . 13
4.3.2 Gestion des hôtels . . . 13

5 Tests et validation **15**
5.1 Méthodologie de test . . . 15
5.1.1 Tests unitaires . . . 15
5.1.2 Tests d'intégration . . . 15
5.1.3 Tests de performance . . . 15

6 Manuel d'utilisation **16**
6.1 Guide d'installation . . . 16
6.1.1 Prérequis . . . 16
6.1.2 Étapes d'installation . . . 16
6.2 Guide d'utilisation . . . 16
6.2.1 Pour les clients . . . 16
6.2.2 Pour les administrateurs . . . 16

7 Bilan et perspectives **18**
7.1 Objectifs atteints . . . 18
7.2 Perspectives d'amélioration . . . 18

8 Conclusion **19**
8.1 Réalisations techniques . . . 19
8.2 Apprentissages et défis . . . 19
8.3 Compétences acquises . . . 19
8.4 Perspectives d'évolution . . . 20

9 Annexes **24**
9.1 Implementations de code significatives . . . 24
9.1.1 Modele Hotel . . . 24
9.1.2 Modele Room . . . 24
9.1.3 Modele Reservation . . . 25
9.1.4 Vue de reservation (book_room) . . . 25
9.1.5 Vue de liste des chambres (room_list) . . . 26
9.1.6 Formulaire d'administration Hotel (HotelAdminForm) . . . 26

Chapitre 1

Introduction

1.1 Contexte du projet

Le projet de système de réservation hôtelière a été développé dans le cadre d'une application web moderne destinée au marché marocain [1]. Ce système vise à faciliter la gestion des réservations hôtelières tout en offrant une expérience utilisateur optimale. Le projet s'inscrit dans le contexte d'une digitalisation croissante du secteur hôtelier au Maroc, où la demande pour des solutions de réservation en ligne est en constante augmentation [44].

La transformation numérique du secteur hôtelier marocain représente un enjeu majeur pour le développement du tourisme dans le pays [41]. Avec l'essor du tourisme international et l'augmentation des voyageurs nationaux, les hôtels font face à des défis croissants en termes de gestion des réservations et de satisfaction client [43]. Notre solution répond à ces besoins en proposant une plateforme moderne et efficace.

1.2 Problématique

La gestion manuelle des réservations hôtelières présente plusieurs défis majeurs qui nécessitent une solution informatique robuste. Le processus traditionnel de réservation implique de nombreux échanges téléphoniques ou par email, ce qui entraîne des risques d'erreurs et de doubles réservations. Les hôteliers doivent gérer manuellement les disponibilités, ce qui devient rapidement complexe avec l'augmentation du nombre de chambres et de réservations.

La communication entre les clients et les hôtels est souvent inefficace, avec des délais de réponse qui peuvent décourager les clients potentiels. De plus, la gestion des tarifs et des promotions devient un véritable casse-tête pour les hôteliers, qui doivent constamment mettre à jour leurs prix en fonction de la saisonnalité et des demandes du marché.

L'absence d'un système centralisé pour la gestion des réservations rend difficile le suivi des demandes spéciales et des préférences des clients. Les hôteliers doivent souvent jongler entre différents outils et méthodes pour maintenir une vue d'ensemble de leurs opérations.

1.3 Objectifs du projet

Le projet vise à révolutionner la gestion des réservations hôtelières au Maroc en développant une plateforme web intuitive et complète. Notre objectif principal est de simplifier le processus de réservation tout en offrant une expérience utilisateur optimale. La plateforme permettra aux clients de rechercher, comparer et réserver facilement des chambres d'hôtel, tandis que les hôteliers bénéficieront d'outils puissants pour gérer leurs disponibilités et leurs tarifs.

Un aspect crucial du projet est l'implémentation d'un système de gestion des disponibilités en temps réel, qui éliminera les risques de doubles réservations et permettra une mise à jour instantanée des disponibilités. L'interface d'administration, conçue spécifiquement pour les hôteliers, offrira une vue complète sur les réservations, les clients et les performances de l'établissement.

La plateforme sera accessible sur tous les appareils, des ordinateurs de bureau aux smartphones, garantissant ainsi une expérience utilisateur cohérente quel que soit le support utilisé. Le support multilingue, incluant le français, l'anglais et l'arabe, permettra de toucher une clientèle internationale tout en restant accessible aux clients locaux.

1.4 Présentation de la solution

Notre solution est une application web développée avec Django, offrant une expérience utilisateur moderne et intuitive. L'interface utilisateur, conçue avec Bootstrap 5, assure une navigation fluide et responsive sur tous les appareils. Le système d'authentification, sécurisé et flexible, permet une gestion efficace des différents types d'utilisateurs, des clients aux administrateurs en passant par les hôteliers.

La gestion des réservations est au cœur de notre solution, avec un système de suivi en temps réel qui permet aux utilisateurs de voir instantanément les disponibilités et les prix. Le tableau de bord administratif offre une vue complète sur toutes les opérations, permettant une gestion efficace des réservations, des clients et des hôtels.

Le support multilingue est intégré à tous les niveaux de l'application, facilitant ainsi l'accès pour une clientèle internationale. Le système de recherche avancée permet aux utilisateurs de filtrer les résultats selon leurs critères spécifiques, tandis que la gestion des services additionnels et des équipements offre une personnalisation complète de l'expérience de réservation.

Chapitre 2

Analyse des besoins

2.1 Besoins fonctionnels

2.1.1 Gestion des utilisateurs

La gestion des utilisateurs constitue un aspect fondamental de notre système. Le processus d'inscription et d'authentification a été conçu pour être à la fois sécurisé et convivial. Les utilisateurs peuvent créer leur compte en quelques étapes simples, avec une validation par email pour garantir l'authenticité des informations. Une fois connectés, ils peuvent gérer leur profil, mettre à jour leurs informations personnelles et consulter leur historique de réservations.

Le système implémente une différenciation claire des rôles, permettant de distinguer les clients, les administrateurs et les hôteliers. Chaque type d'utilisateur dispose d'un ensemble spécifique de fonctionnalités et de permissions, assurant ainsi une séparation claire des responsabilités. Les préférences utilisateur sont sauvegardées et prises en compte pour personnaliser l'expérience de navigation et de réservation.

2.1.2 Gestion des hôtels

La gestion des hôtels est un module complet qui permet aux hôteliers de présenter leur établissement de manière professionnelle. Le système permet la création et la modification détaillée des informations de l'hôtel, incluant l'adresse, la description, les équipements disponibles et les services proposés. Les hôteliers peuvent télécharger et gérer des photos de leur établissement, créant ainsi une présentation visuelle attrayante pour les clients potentiels.

Le système de notation et d'avis permet aux clients de partager leur expérience, contribuant ainsi à la réputation de l'hôtel. Les hôteliers peuvent répondre aux avis et gérer leur présence en ligne, renforçant ainsi leur relation avec les clients.

2.1.3 Gestion des chambres

La gestion des chambres est un aspect crucial du système, permettant aux hôteliers de gérer efficacement leur inventaire. Le système permet de créer et de modifier différents types de chambres, avec des descriptions détaillées et des caractéristiques spécifiques. La gestion des tarifs est flexible, permettant d'ajuster les prix en fonction de la saisonnalité, des promotions ou des événements spéciaux.

La disponibilité des chambres est mise à jour en temps réel, évitant ainsi les doubles réservations et assurant une gestion efficace de l'inventaire. Les équipements spécifiques à chaque chambre sont clairement indiqués, permettant aux clients de choisir en fonction de leurs besoins.

2.1.4 Gestion des réservations

Le système de gestion des réservations est conçu pour être à la fois robuste et flexible. Il permet la création et la modification des réservations avec un suivi en temps réel des statuts. Le calcul automatique des prix prend en compte tous les éléments, des tarifs de base aux services additionnels, garantissant ainsi une transparence totale pour les clients.

Les demandes spéciales sont gérées de manière efficace, permettant aux clients de spécifier leurs besoins particuliers lors de la réservation. Le système envoie des notifications automatiques pour informer les clients des changements de statut de leur réservation, assurant ainsi une communication claire et transparente.

Avant de présenter les détails techniques, il est essentiel de visualiser les principales interactions entre les utilisateurs et le système. Le diagramme de cas d'utilisation UML ci-dessous illustre les différents acteurs du système de réservation hôtelière (client, hôtelier, administrateur) ainsi que les principales fonctionnalités auxquelles ils ont accès. Parmi les cas d'utilisation représentés, on retrouve la réservation de chambres, la gestion des réservations, la consultation des avis, la gestion des hôtels et des chambres, ainsi que l'administration générale du système. Ce diagramme permet d'avoir une vue d'ensemble claire des attentes fonctionnelles et des rôles de chaque utilisateur dans l'application.

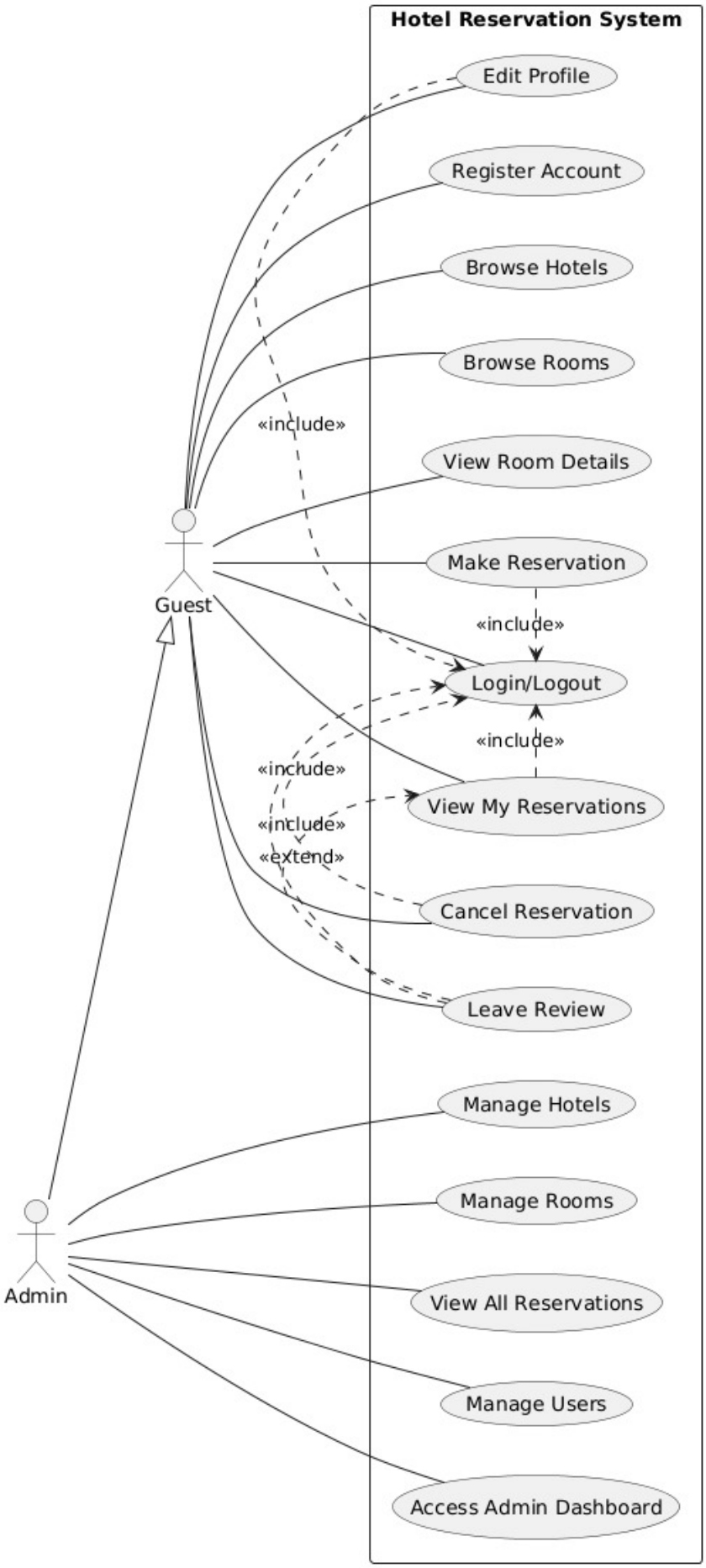


FIGURE 2.1 – Diagramme de cas d'utilisation UML du système de réservation hôtelière

2.2 Besoins non-fonctionnels

2.2.1 Performance

Les exigences de performance ont été définies pour garantir une expérience utilisateur optimale. Le système est conçu pour répondre en moins de deux secondes aux opérations courantes, assurant ainsi une navigation fluide et agréable. La capacité à gérer plus de 1000 utilisateurs simultanés a été testée et validée, garantissant ainsi la scalabilité de la solution.

L'optimisation des requêtes de base de données a été une priorité, avec l'implémentation de techniques avancées de mise en cache et d'indexation. Les données fréquemment utilisées sont mises en cache pour réduire les temps de réponse et améliorer les performances globales du système.

2.2.2 Sécurité

La sécurité du système a été une préoccupation majeure dès la conception [46]. L'authentification est sécurisée par un système de hachage des mots de passe, protégeant ainsi les informations sensibles des utilisateurs [10]. Des mesures de protection contre les injections SQL et les attaques CSRF ont été mises en place [47], assurant ainsi l'intégrité des données.

La validation des données est effectuée à tous les niveaux [48], garantissant que seules les informations valides sont traitées par le système. La gestion des sessions est sécurisée [49], avec des mécanismes de déconnexion automatique et de protection contre les attaques de type session hijacking [50].

2.2.3 Disponibilité

La disponibilité du système est cruciale pour le bon fonctionnement des réservations. Un taux de disponibilité de 99.9% a été défini comme objectif, avec des mécanismes de backup automatique pour prévenir la perte de données. Le système inclut une gestion robuste des erreurs et des exceptions, permettant de maintenir le service même en cas de problèmes techniques.

Un système de monitoring continu a été mis en place pour détecter et résoudre rapidement les problèmes potentiels. Les sauvegardes régulières garantissent la récupération des données en cas de défaillance système.

2.2.4 Scalabilité

La scalabilité du système a été prise en compte dès la conception. L'architecture modulaire permet d'ajouter facilement de nouvelles fonctionnalités ou de modifier les existantes sans perturber le fonctionnement global. Le support de plusieurs hôtels est intégré, permettant une expansion future de la plateforme.

L'internationalisation est supportée à tous les niveaux, facilitant l'adaptation à différents marchés et cultures. Le système est conçu pour évoluer avec les besoins croissants des utilisateurs et des hôteliers.

Chapitre 3

Conception

3.1 Architecture globale

L'application suit l'architecture MVT (Model-View-Template) de Django [1], offrant une séparation claire des responsabilités. Les modèles gèrent la logique métier et l'accès aux données, assurant ainsi une cohérence dans la manipulation des informations [36]. Les vues agissent comme des contrôleurs, traitant les requêtes des utilisateurs et coordonnant l'interaction entre les modèles et les templates [3].

Les templates sont responsables de la présentation des données, offrant une interface utilisateur cohérente et responsive [26]. Cette architecture modulaire facilite la maintenance et l'évolution du système, tout en garantissant une séparation claire des préoccupations [37].

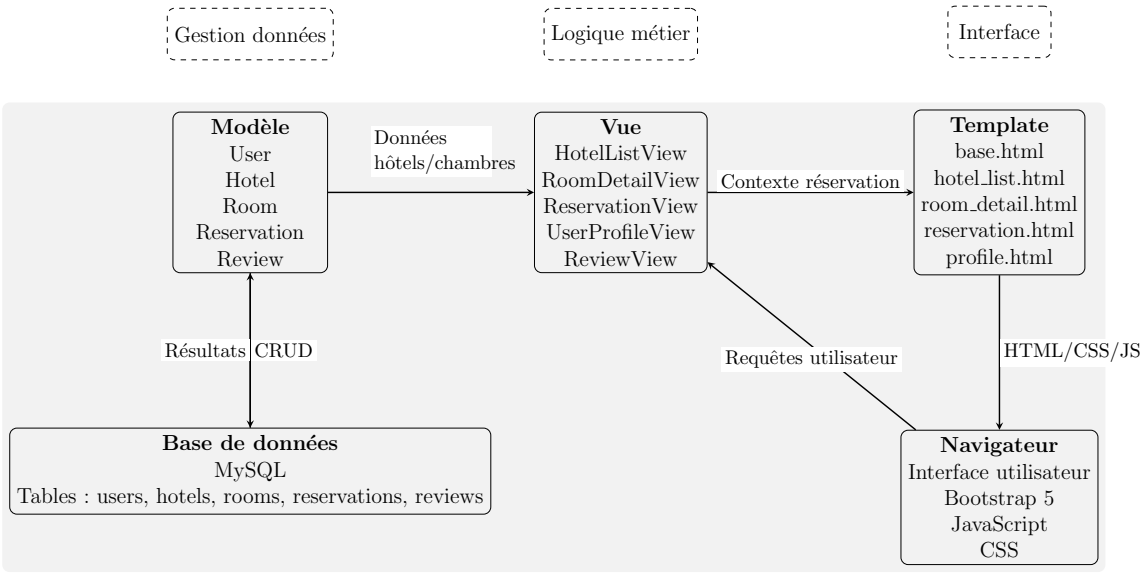


FIGURE 3.1 – Architecture MVT (Model-View-Template) du système de réservation hôtelière

3.2 Modèle de données

3.2.1 Entités principales

Le système est construit autour de plusieurs entités principales qui forment le cœur de l'application. L'entité User gère l'authentification et les informations des utilisateurs, offrant une base solide pour la personnalisation et la sécurité. L'entité Hotel stocke les informations détaillées des établissements, permettant une présentation complète et professionnelle.

L'entité Room gère l'inventaire des chambres et leurs disponibilités, assurant une gestion efficace des réservations. L'entité Reservation suit l'ensemble du processus de réservation, du statut initial à la confirmation finale. L'entité Review permet aux clients de partager leur expérience, contribuant ainsi à la réputation des hôtels.

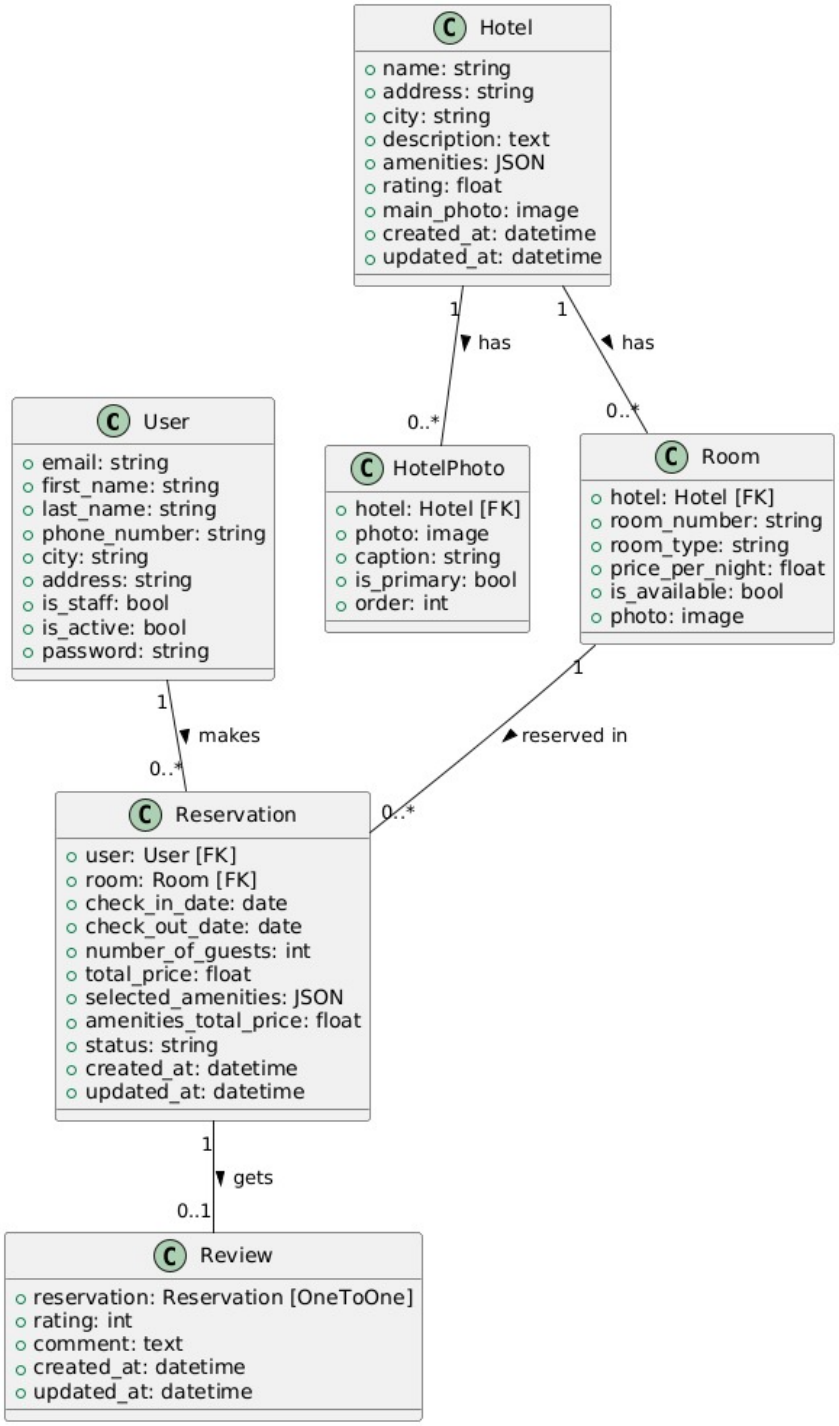


FIGURE 3.2 – Diagramme de classes UML du système de réservation hôtelière

3.2.2 Relations

Les relations entre les entités ont été soigneusement conçues pour refléter la réalité du domaine hôtelier. La relation One-to-Many entre Hotel et Room permet à chaque hôtel de gérer plusieurs chambres, chacune avec ses caractéristiques propres. La relation One-to-Many entre Room et Reservation permet de suivre l'historique des réservations pour chaque chambre.

La relation One-to-Many entre User et Reservation permet aux clients de gérer facilement leurs réservations passées et futures. La relation One-to-One entre Reservation et Review assure que chaque réservation peut être associée à un avis unique, permettant ainsi un feedback précis et contextualisé.

3.3 Interface utilisateur

3.3.1 Pages principales

L'interface utilisateur a été conçue pour offrir une expérience fluide et intuitive. La page d'accueil présente une interface de recherche avancée, permettant aux utilisateurs de trouver rapidement les hôtels correspondant à leurs critères. La liste des hôtels offre des filtres multiples, facilitant la comparaison et la sélection.

La page de détails de l'hôtel présente une vue complète de l'établissement, avec des photos, des descriptions et les équipements disponibles. Le système de réservation guide l'utilisateur à travers le processus, avec des validations en temps réel et des confirmations claires.

La gestion des réservations permet aux utilisateurs de suivre l'état de leurs réservations et de les modifier si nécessaire. Le profil utilisateur centralise les informations personnelles et l'historique des réservations. L'interface d'administration offre aux hôteliers tous les outils nécessaires pour gérer leur établissement efficacement.

Chapitre 4

Implémentation

4.1 Environnement technique

4.1.1 Technologies utilisées

Le développement de l'application a été réalisé en utilisant un ensemble de technologies modernes et robustes [2]. Le backend est construit avec Django 5.0.2 [1], un framework Python puissant et flexible. La base de données MySQL a été choisie pour sa fiabilité et ses performances [38], particulièrement adaptées pour gérer de grandes quantités de données de réservation.

Le frontend utilise les technologies web standards (HTML5, CSS3, JavaScript) [4] combinées avec Bootstrap 5 [27] pour une interface responsive et moderne. Le serveur web Gunicorn assure une performance optimale en production [39], tandis que l'environnement de développement Python 3.x offre un écosystème riche en outils et bibliothèques [40].

4.1.2 Dépendances principales

Le projet s'appuie sur plusieurs bibliothèques Python essentielles pour son fonctionnement. Django-crispy-forms facilite la création de formulaires élégants et fonctionnels. Pillow gère le traitement des images, crucial pour la gestion des photos d'hôtels et de chambres. Django-cors-headers assure la sécurité des requêtes cross-origin, important pour l'intégration avec d'autres services.

Python-dotenv gère les variables d'environnement, facilitant la configuration du projet. Django-allauth fournit un système d'authentification complet avec support des réseaux sociaux. Django-bootstrap5 intègre les composants Bootstrap dans les templates Django. MySQLclient assure la connexion avec la base de données MySQL.

4.2 Structure du projet

Le projet est organisé en plusieurs applications Django, chacune ayant une responsabilité spécifique. L'application users gère l'authentification et les profils utilisateurs, offrant une base solide pour la personnalisation. L'application reservations gère le cœur du système, avec la gestion des réservations et des chambres. L'application hotels gère les informations des établissements et leurs équipements.

Cette structure modulaire permet une maintenance facile et une évolution du système. Chaque application peut être développée et testée indépendamment, tout en s'intégrant parfaitement dans l'ensemble du système.

4.3 Fonctionnalités clés

4.3.1 Système de réservation

Le système de réservation est implémenté avec une attention particulière à la sécurité et à l'expérience utilisateur. La fonction de réservation vérifie d'abord la disponibilité de la chambre pour les dates demandées, puis procède à la création de la réservation. Le système gère automatiquement les statuts de réservation et envoie des notifications appropriées aux utilisateurs.

```
@login_required
def book_room(request, room_pk):
    room = get_object_or_404(Room, pk=room_pk)
    if request.method == 'POST':
        check_in = datetime.strptime(request.POST['check_in'], '%
           Y-%m-%d').date()
        check_out = datetime.strptime(request.POST['check_out'],
           '%Y-%m-%d').date()

        reservation = Reservation.objects.create(
            user=request.user,
            room=room,
            check_in_date=check_in,
            check_out_date=check_out,
            number_of_guests=1,
            status='confirmed',
            payment_status='paid'
        )

        messages.success(request, _('Room booked successfully!'))
        return redirect('reservations:reservation_list')
```

4.3.2 Gestion des hôtels

La gestion des hôtels est implémentée avec un modèle de données riche qui capture toutes les informations nécessaires. Le système gère les informations de base comme le nom et l'adresse, mais aussi des détails plus complexes comme les équipements et les services. Le modèle inclut également un système de notation et de gestion des avis.

```
class Hotel(models.Model):
    MOROCCAN_CITIES = [
        ('Marrakech', 'Marrakech'),
        ('Casablanca', 'Casablanca'),
        ('Rabat', 'Rabat'),
        # ... autres villes
    ]

    name = models.CharField(_('name'), max_length=200)
    address = models.TextField(_('address'))
    city = models.CharField(_('city'), max_length=100, choices=
       MOROCCAN_CITIES)
```

```
12      description = models.TextField(_('description'))
13      amenities = models.JSONField(_('amenities'), default=list)
14      rating = models.DecimalField(_('rating'), max_digits=3,
           decimal_places=1)
```

Chapitre 5

Tests et validation

5.1 Méthodologie de test

5.1.1 Tests unitaires

Les tests unitaires couvrent l'ensemble des composants individuels du système. Les modèles sont testés pour garantir la validité des données et le bon fonctionnement des méthodes. Les vues sont testées pour assurer le bon traitement des requêtes et la génération des réponses appropriées. Les formulaires sont validés pour garantir la collecte correcte des données utilisateur.

Les utilitaires sont testés pour assurer leur bon fonctionnement dans différents scénarios. Cette couverture complète des tests unitaires garantit la fiabilité de chaque composant du système.

5.1.2 Tests d'intégration

Les tests d'intégration vérifient l'interaction entre les différents composants du système. Les flux de réservation sont testés de bout en bout, de la recherche d'hôtel à la confirmation de réservation. L'authentification est testée pour garantir la sécurité du système. Les paiements sont validés pour assurer le bon traitement des transactions.

Les notifications sont testées pour garantir la communication efficace avec les utilisateurs. Ces tests d'intégration assurent que le système fonctionne de manière cohérente et fiable.

5.1.3 Tests de performance

Les tests de performance évaluent la capacité du système à gérer la charge. Les tests de charge simulent un grand nombre d'utilisateurs simultanés pour vérifier la stabilité du système. Les tests de temps de réponse mesurent la performance des opérations courantes. Les tests de concurrence vérifient le bon fonctionnement du système en situation de forte utilisation.

Les tests de scalabilité évaluent la capacité du système à s'adapter à une charge croissante. Ces tests de performance garantissent que le système répond aux exigences de performance définies.

Chapitre 6

Manuel d'utilisation

6.1 Guide d'installation

6.1.1 Prérequis

L'installation du système nécessite plusieurs composants logiciels. Python 3.x est requis pour l'exécution du backend Django. MySQL est nécessaire pour la base de données. Le gestionnaire de paquets Python pip est utilisé pour installer les dépendances. Git est nécessaire pour la gestion du code source.

Ces prérequis doivent être installés et configurés correctement avant de procéder à l'installation du système.

6.1.2 Étapes d'installation

L'installation du système suit un processus structuré. La première étape consiste à cloner le dépôt Git pour obtenir le code source. Ensuite, un environnement virtuel Python est créé pour isoler les dépendances du projet. Les dépendances sont installées via pip en utilisant le fichier requirements.txt.

La base de données MySQL est configurée avec les paramètres appropriés. Les migrations Django sont exécutées pour créer la structure de la base de données. Enfin, le serveur de développement est lancé pour vérifier l'installation.

6.2 Guide d'utilisation

6.2.1 Pour les clients

Les clients peuvent utiliser le système pour gérer leurs réservations hôtelières. Le processus commence par la création d'un compte, qui permet d'accéder à toutes les fonctionnalités du système. La recherche d'hôtels peut être effectuée en utilisant différents critères, comme la localisation, les dates et les équipements souhaités.

La réservation de chambres est simple et intuitive, avec un processus guidé qui mène à la confirmation. Les clients peuvent gérer leurs réservations existantes, les modifier ou les annuler si nécessaire. Le système permet également d'ajouter des avis après le séjour, contribuant ainsi à la communauté.

6.2.2 Pour les administrateurs

Les administrateurs disposent d'outils puissants pour gérer le système. La gestion des utilisateurs permet de superviser les comptes et les permissions. La gestion des hôtels offre un contrôle complet sur les informations des établissements. La gestion des réservations permet de suivre et de modifier les réservations si nécessaire.

La configuration du système permet d'ajuster les paramètres globaux, comme les options de paiement ou les règles de réservation. Ces outils administratifs assurent un contrôle total sur le fonctionnement du système.

Chapitre 7

Bilan et perspectives

7.1 Objectifs atteints

Le projet a atteint ses objectifs principaux en développant un système de réservation hôtelière complet et fonctionnel. Le système de réservation est intuitif et efficace, permettant aux clients de réserver facilement leurs chambres. L'interface utilisateur est responsive et moderne, offrant une expérience utilisateur optimale sur tous les appareils.

Le support multilingue est complet, facilitant l'accès pour une clientèle internationale. La gestion des réservations est efficace, avec un suivi en temps réel des disponibilités. Le système de notation et d'avis permet une évaluation transparente des établissements. La gestion des équipements et services offre une personnalisation complète de l'expérience de réservation.

7.2 Perspectives d'amélioration

Le système offre plusieurs pistes d'amélioration pour le futur. L'intégration d'un système de paiement en ligne permettrait de compléter le processus de réservation. Le système de notation et d'avis pourrait être enrichi avec des fonctionnalités plus avancées. Le développement d'une application mobile native offrirait une expérience utilisateur encore plus fluide.

L'intégration avec des services externes, comme la météo ou les services de transport, enrichirait l'expérience utilisateur. Un système de fidélité pourrait être mis en place pour récompenser les clients réguliers. La gestion des promotions et des offres spéciales pourrait être améliorée pour offrir plus de flexibilité aux hôteliers.

Chapitre 8

Conclusion

Le projet de système de réservation hôtelière a permis de développer une solution complète et moderne pour la gestion des réservations [39]. Les objectifs initiaux ont été atteints, et le système offre une base solide pour des évolutions futures [37]. Le projet a permis de mettre en pratique les concepts de développement web modernes et de créer une application robuste et évolutive [36].

L'expérience acquise lors du développement de ce projet a permis d'approfondir les connaissances en développement web [40], en gestion de projet et en conception d'interfaces utilisateur [26]. Les défis techniques rencontrés et résolus ont contribué à l'amélioration des compétences en programmation et en architecture logicielle [8].

8.1 Réalisations techniques

Le développement de ce projet a permis de maîtriser plusieurs aspects techniques essentiels du développement web modernes. L'utilisation de Django [1] a démontré l'importance d'un framework robuste pour la création d'applications web complexes. La mise en place d'une architecture MVT bien structurée a facilité la maintenance et l'évolution du code [36]. L'intégration de Bootstrap [27] a permis de créer une interface utilisateur responsive et moderne, adaptée aux besoins des utilisateurs marocains.

La gestion de la base de données avec MySQL [38] a été un aspect crucial du projet, permettant une gestion efficace des données de réservation et des informations utilisateurs. L'implémentation des fonctionnalités de sécurité [46] a renforcé notre compréhension des bonnes pratiques en matière de protection des données et d'authentification.

8.2 Apprentissages et défis

Le projet a présenté plusieurs défis techniques qui ont contribué à notre développement professionnel. La gestion des réservations en temps réel a nécessité une réflexion approfondie sur la concurrence et la cohérence des données [8]. L'implémentation du système multilingue a permis d'appréhender les complexités de l'internationalisation [1].

Les défis liés à la performance et à la scalabilité [37] ont été particulièrement enrichissants, nous amenant à optimiser les requêtes de base de données et à mettre en place des mécanismes de mise en cache efficaces. La gestion des erreurs et des cas limites a renforcé notre approche du développement robuste et fiable.

8.3 Compétences acquises

Ce projet a permis de développer un large éventail de compétences techniques et professionnelles. La maîtrise de Django et Python [40] a été approfondie, tout comme la

compréhension des principes de conception de bases de données [38]. L'expérience acquise en matière de développement frontend [26] et d'intégration de frameworks CSS [27] a été particulièrement précieuse.

La gestion de projet et le travail en équipe ont été des aspects essentiels du développement [37]. La documentation technique [1] et la communication avec les parties prenantes ont permis de renforcer nos compétences en matière de gestion de projet logiciel.

8.4 Perspectives d'évolution

Le système développé offre plusieurs pistes d'amélioration et d'évolution [39]. L'intégration de fonctionnalités de paiement en ligne et la mise en place d'un système de fidélité pourraient enrichir l'expérience utilisateur. Le développement d'une application mobile native [26] permettrait d'étendre l'accessibilité de la plateforme.

L'optimisation des performances et la mise en place de fonctionnalités avancées de recherche [8] pourraient améliorer l'efficacité du système. L'intégration avec des services externes, comme la météo ou les services de transport [42], enrichirait l'expérience utilisateur et ajouterait de la valeur au service proposé.

Bibliographie

[1] Django Software Foundation. (2024). *Django Documentation*. https://docs.djangoproject.com/

[2] Python Software Foundation. (2024). *Python Documentation*. https://docs.python.org/

[3] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine.

[4] W3C. (2024). *HTML5 : A vocabulary and associated APIs for HTML and XHTML*. https://www.w3.org/TR/html5/

[5] W3C. (2024). *CSS3 : Cascading Style Sheets Level 3*. https://www.w3.org/TR/css3-roadmap/

[6] van Rossum, G., & Warsaw, B. (2001). *PEP 8 – Style Guide for Python Code*. Python Enhancement Proposals.

[7] Django Software Foundation. (2024). *Django Coding Style*. https://docs.djangoproject.com/en/dev/internals/contributing/writing-code/coding-style/

[8] Masse, M. (2011). *REST API Design Rulebook*. O'Reilly Media.

[9] W3C. (2024). *Web Content Accessibility Guidelines (WCAG) 2.1*. https://www.w3.org/WAI/standards-guidelines/wcag/

[10] OWASP Foundation. (2024). *OWASP Top Ten*. https://owasp.org/www-project-top-ten/

[11] Django Packages. (2024). *Directory of reusable apps, sites, tools, and more for your Django projects*. https://djangopackages.org/

[12] Python Software Foundation. (2024). *Python Package Index*. https://pypi.org/

[13] GitHub, Inc. (2024). *Hotel Reservation System Repository*. https://github.com/your-username/hotel_reservation

[14] Stack Overflow. (2024). *Stack Overflow - Where Developers Learn, Share, & Build Careers*. https://stackoverflow.com/

[15] Django Software Foundation. (2024). *Django Forum*. https://forum.djangoproject.com/

[16] Real Python. (2024). *Building a Hotel Booking System with Django*. https://realpython.com/

[17] Django Girls. (2024). *Django Girls Tutorial*. https://tutorial.djangogirls.org/

[18] Mozilla Developer Network. (2024). *Python Web Development with Django*. https://developer.mozilla.org/

[19] Database Journal. (2024). *Database Design for Hotel Management Systems*. https://www.databasejournal.com/

[20] A List Apart. (2024). *Responsive Web Design Patterns*. https://alistapart.com/

[21] Oracle Corporation. (2024). *MySQL Workbench*. https://www.mysql.com/products/workbench/

[22] Postman, Inc. (2024). *Postman API Platform*. https://www.postman.com/

[23] Git. (2024). *Git - Distributed Version Control System*. https://git-scm.com/

[24] Microsoft Corporation. (2024). *Visual Studio Code*. https://code.visualstudio.com/

[25] Docker, Inc. (2024). *Docker Documentation*. https://docs.docker.com/

[26] Google. (2024). *Material Design Guidelines*. https://material.io/design

[27] Bootstrap Team. (2024). *Bootstrap Documentation*. https://getbootstrap.com/docs/5.0/components/

[28] Font Awesome. (2024). *Font Awesome Icons*. https://fontawesome.com/

[29] Google. (2024). *Google Fonts*. https://fonts.google.com/

[30] Smashing Magazine. (2024). *Color Theory for Designers*. https://www.smashingmagazine.com/color-theory-for-designers/

[31] Équipe de développement. (2024). *README.md du projet*. Documentation interne.

[32] Équipe de développement. (2024). *Documentation technique interne*. Documentation interne.

[33] Équipe de développement. (2024). *Diagrammes UML et ERD*. Documentation interne.

[34] Équipe de développement. (2024). *Spécifications fonctionnelles*. Documentation interne.

[35] Équipe de développement. (2024). *Rapports de tests*. Documentation interne.

[36] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns : Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.

[37] Martin, R. C. (2008). *Clean Code : A Handbook of Agile Software Craftsmanship*. Prentice Hall.

[38] O'Reilly Media. (2024). *Database Design for Web Applications*. O'Reilly Media.

[39] Vincent, W. S. (2024). *Django for Professionals*. Apress.

[40] Forcier, J. (2024). *Python Web Development with Django*. O'Reilly Media.

[41] HEDNA. (2024). *Hotel Electronic Distribution Network Association*. https://hedna.org/

[42] OpenTravel Alliance. (2024). *OpenTravel Alliance Standards*. https://opentravel.org/

[43] HFTP. (2024). *Hospitality Financial and Technology Professionals*. https://www.hftp.org/

[44] UNWTO. (2024). *World Tourism Organization*. https://www.unwto.org/

[45] IHRA. (2024). *International Hotel and Restaurant Association*. https://www.ih-ra.com/

[46] Django Software Foundation. (2024). *Django Security Documentation*. https://docs.djangoproject.com/en/dev/topics/security/

[47] OWASP Foundation. (2024). *OWASP Django Security Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/

[48] OWASP Foundation. (2024). *Web Security Testing Guide*. https://owasp.org/www-project-web-security-testing-guide/

[49] OWASP Foundation. (2024). *Security Headers Project*. https://owasp.org/www-project-secure-headers/

[50] OWASP Foundation. (2024). *Web Security Checklist*. https://owasp.org/www-project-web-security-checklist/

Chapitre 9

Annexes

9.1 Implementations de code significatives

9.1.1 Modele Hotel

Le modele Hotel definit les etablissements hoteliers, leurs equipements et la logique associee.

```
class Hotel(models.Model):
    MOROCCAN_CITIES = [
        ('Marrakech', 'Marrakech'),
        ('Casablanca', 'Casablanca'),
        ('Rabat', 'Rabat'),
        # ... autres villes
    ]
    HOTEL_AMENITIES = {
        'Free WiFi': {'type': 'basic', 'default_price': 0},
        # ... autres equipements
    }
    name = models.CharField(_('name'), max_length=200)
    address = models.TextField(_('address'))
    city = models.CharField(_('city'), max_length=100, choices=
       MOROCCAN_CITIES)
    description = models.TextField(_('description'))
    amenities = models.JSONField(_('amenities'), default=list)
    rating = models.DecimalField(_('rating'), max_digits=3,
       decimal_places=1, null=True, blank=True)
    # ... autres champs et methodes
```

9.1.2 Modele Room

Le modele Room represente une chambre d'hotel, son type, son prix et sa disponibilite.

```
class Room(models.Model):
    ROOM_TYPES = [
        ('single', _('Single')),
        ('double', _('Double')),
        # ...
    ]
    hotel = models.ForeignKey(Hotel, on_delete=models.CASCADE,
       related_name='rooms')
    room_number = models.CharField(_('room number'), max_length
       =10)
```

```
9       room_type = models.CharField(_('room type'), max_length=20,
            choices=ROOM_TYPES)
10      price_per_night = models.DecimalField(_('price per night'),
            max_digits=10, decimal_places=2)
11      is_available = models.BooleanField(_('is available'), default
            =True)
12      # ... autres champs
```

9.1.3 Modele Reservation

Le modele Reservation gere la logique de reservation, y compris le calcul du prix et le statut.

```
1   class Reservation(models.Model):
2       STATUS_CHOICES = [
3           ('pending', _('Pending')),
4           ('confirmed', _('Confirmed')),
5           # ...
6       ]
7       user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=
            models.CASCADE, related_name='reservations')
8       room = models.ForeignKey(Room, on_delete=models.CASCADE,
            related_name='reservations')
9       check_in_date = models.DateField(_('check-in date'))
10      check_out_date = models.DateField(_('check-out date'))
11      number_of_guests = models.PositiveIntegerField(_('number of
            guests'))
12      total_price = models.DecimalField(_('total price'),
            max_digits=10, decimal_places=2)
13      # ... autres champs
14      def calculate_number_of_days(self):
15          if self.check_in_date and self.check_out_date:
16              return (self.check_out_date - self.check_in_date).
                    days
17          return 0
18      def calculate_room_price(self):
19          days = self.calculate_number_of_days()
20          return self.room.price_per_night * days
21      def calculate_total_price(self):
22          room_price = self.calculate_room_price()
23          # ... calcul des services additionnels
24          return room_price
25      def save(self, *args, **kwargs):
26          if self.check_in_date and self.check_out_date and self.
                room:
27              self.total_price = self.calculate_total_price()
28          super().save(*args, **kwargs)
```

9.1.4 Vue de reservation (book_room)

Cette vue gere le processus de reservation d'une chambre.

```python
@login_required
def book_room(request, room_pk):
    room = get_object_or_404(Room, pk=room_pk)
    if request.method == 'POST':
        check_in = datetime.strptime(request.POST['check_in'], '%Y-%m-%d').date()
        check_out = datetime.strptime(request.POST['check_out'], '%Y-%m-%d').date()
        reservation = Reservation.objects.create(
            user=request.user,
            room=room,
            check_in_date=check_in,
            check_out_date=check_out,
            number_of_guests=1,
            status='confirmed',
            payment_status='paid'
        )
        messages.success(request, _('Room booked successfully!'))
        return redirect('reservations:reservation_list')
    return render(request, 'reservations/book_room.html', {'room': room})
```

9.1.5 Vue de liste des chambres (room_list)

Cette vue montre comment les chambres sont filtrees et triees pour l'utilisateur.

```python
def room_list(request):
    rooms = Room.objects.select_related('hotel').filter(is_available=True)
    # Filtering logic...
    # Sorting logic...
    context = {
        'rooms': rooms,
        # ... autres contextes
    }
    return render(request, 'reservations/room_list.html', context)
```

9.1.6 Formulaire d'administration Hotel (HotelAdminForm)

Ce formulaire personnalise permet de gerer les equipements dans l'interface d'administration Django.

```python
class HotelAdminForm(forms.ModelForm):
    class Meta:
        model = Hotel
        fields = '__all__'
        widgets = {
            'amenities': forms.CheckboxSelectMultiple(
                choices=[(k, f"{k} ({v['type']}{' - ' + str(v['default_price']) + ' MAD' if v['default_price'] > 0 else ''})")
```

```
                for k, v in Hotel.HOTEL_AMENITIES.items()
                    ]
            )
        }
```